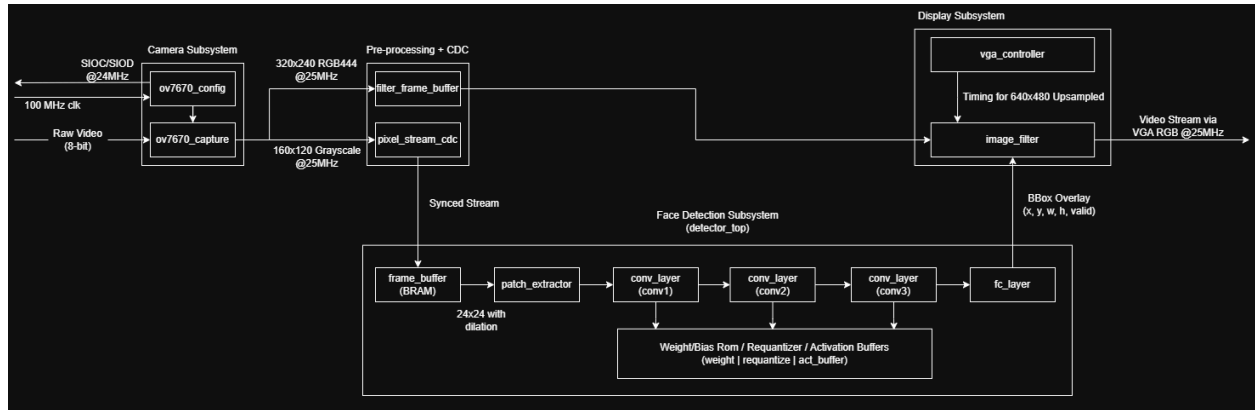


Group Member

Group name : บ้านพิบช้าง

Name	Student Number
Jiruchya Niamluang	6731307721
Phonlakrit Lookyee	6731336921
Raweewan Niyomkullasit	6731344921

Overall Design Block Diagram



Original picture : [Block Design.drawio](#)

Design Decision.

Module Architecture

- Design: Decouple Dataflow Architecture, which separates the high-bandwidth video display path from the high-latency neural network (NN) path.
- Reason: separating 2 datapaths enables parallelism and avoids different latencies in each datapath.
 - Display Path: passthrough to provide visual feedback
 - Detection Path: downsampled grayscale buffer to reduce computational load.

Communication Protocols

- Decision - SCCB
- Reason: Because it is compatible with OV sensors, which take a low pin count to control internal camera parameters.

Clock and Data Transfer Rate

- Clock Frequency Selection:
 - 100MHz (Basys3 Clock): Used for NN to minimize computation time
 - 24-25MHz (Camera/VGA): Used for VGA @60Hz and standard OV7670 timing requirements.
- Constraints & Timing:
 - Clock Domain Crossing (CDC): pixel_stream_cdc.v module handles the transition from the camera's asynchronous pclk to the FPGA's 100 MHz, preventing metastability.

Resource Utilization

- BRAM: uses in frame_buffer and act_buffer, which keeps the LUT usage low
- Sequential MAC: uses in conv_layer, which is slower than going for a parallel approach, but it is much smaller

Implementation Detail

Module : camera_vga_top

Module code :

```
`timescale 1ns / 1ps
`include "scales.vh"
// top module: OV7670 camera + NN face detector + VGA display
// pipeline: camera → capture → CDC → detector → face_* → VGA bounding-box overlay

module camera_vga_top(
    input  wire      clk,           // 100 MHz from Basys 3
    input  wire      reset,        // BTNC, async active-high

    // OV7670 interface
    input  wire [7:0] camera_data,
    input  wire      camera_href,
    input  wire      camera_vsync,
    input  wire      camera_pclk,
    output wire      camera_xclk,
    output wire      camera_pwdn,
    output wire      camera_reset,
    inout  wire      camera_siod,
    output wire      camera_sioc,

    // VGA
    output wire      vga_hsync,
    output wire      vga_vsync,
    output wire [3:0] vga_red,
    output wire [3:0] vga_green,
    output wire [3:0] vga_blue,

    // Operator switches / LEDs
    input  wire [2:0] filter_sel,
    output wire [7:0] led
);

// clocks
wire clk_25mhz;
wire clk_24mhz;
wire clk_locked;

clk_wiz_0 clk_gen (
    .clk_in1(clk),
    .clk_out1(clk_25mhz),
    .clk_out2(clk_24mhz),
    .reset(reset),
    .locked(clk_locked)
```

```

);

// reset synchronizers (async-assert, sync-deassert)
reg [1:0] rst_clk_sync;
always @(posedge clk or posedge reset) begin
    if (reset) rst_clk_sync <= 2'b11;
    else      rst_clk_sync <= {rst_clk_sync[0], 1'b0};
end
wire rst_clk      = rst_clk_sync[1] | ~clk_locked;

reg [1:0] rst_25_sync;
always @(posedge clk_25mhz or posedge reset) begin
    if (reset) rst_25_sync <= 2'b11;
    else      rst_25_sync <= {rst_25_sync[0], 1'b0};
end
wire rst_25 = rst_25_sync[1];

// camera strapping
assign camera_pwdn  = 1'b0;          // not in power-down
assign camera_reset = 1'b1;          // reset deasserted (active low)
assign camera_xclk  = clk_24mhz;

// SCCB config
wire config_done;
wire sccb_busy;
wire sccb_nak_seen;

ov7670_config camera_config (
    .clk(clk),
    .reset(reset || !clk_locked),
    .sioc(camera_sioc),
    .siod(camera_siod),
    .config_done(config_done),
    .sccb_busy(sccb_busy),
    .sccb_nak_seen(sccb_nak_seen)
);

// camera capture
wire [16:0] write_addr;
wire [11:0] write_data;
wire      write_enable;

// pclk grayscale stream
wire      cam_stream_valid;
wire      cam_stream_frame_start;
wire      cam_stream_line_start;
wire [7:0] cam_stream_pixel;

// capture diagnostics (pclk domain)

```

```

wire      cap_frame_format_ok;
wire      cap_frame_heartbeat;

ov7670_capture camera_capture (
    .pclk(camera_pclk),
    .vsync(camera_vsync),
    .href(camera_href),
    .data_in(camera_data),
    .reset(reset || !config_done),

    .frame_addr(write_addr),
    .frame_pixel(write_data),
    .frame_we(write_enable),

    .stream_valid(cam_stream_valid),
    .stream_frame_start(cam_stream_frame_start),
    .stream_line_start(cam_stream_line_start),
    .stream_pixel(cam_stream_pixel),

    .frame_format_ok(cap_frame_format_ok),
    .frame_heartbeat(cap_frame_heartbeat)
);

// CDC: pclk diagnostics → clk domain (for LEDs)
(* ASYNC_REG = "TRUE" *) reg cap_fok_meta;
(* ASYNC_REG = "TRUE" *) reg cap_fok_sync;
(* ASYNC_REG = "TRUE" *) reg cap_fhb_meta;
(* ASYNC_REG = "TRUE" *) reg cap_fhb_sync;
always @(posedge clk) begin
    cap_fok_meta <= cap_frame_format_ok;
    cap_fok_sync <= cap_fok_meta;
    cap_fhb_meta <= cap_frame_heartbeat;
    cap_fhb_sync <= cap_fhb_meta;
end

// VGA timing
wire      vga_active;
wire [9:0] vga_x;
wire [9:0] vga_y;

vga_controller vga_ctrl (
    .clk(clk_25mhz),
    .reset(rst_25),
    .hsync(vga_hsync),
    .vsync(vga_vsync),
    .active(vga_active),
    .x_pos(vga_x),
    .y_pos(vga_y)
);

```

```

// VGA read address (2x pixel doubling; timing delayed 1 cycle to match BRAM latency)
wire [16:0] read_addr;
wire [11:0] read_data;
wire [16:0] row_offset = (vga_y[9:1] << 8) + (vga_y[9:1] << 6); // y/2 * 320
assign read_addr = row_offset + vga_x[9:1];

reg          vga_active_d;
reg [9:0]    vga_x_d;
reg [9:0]    vga_y_d;
always @(posedge clk_25mhz) begin
    if (rst_25) begin
        vga_active_d <= 1'b0;
        vga_x_d      <= 10'd0;
        vga_y_d      <= 10'd0;
    end else begin
        vga_active_d <= vga_active;
        vga_x_d      <= vga_x;
        vga_y_d      <= vga_y;
    end
end

// RGB444 frame buffer
filter_frame_buffer fb (
    .clka(camera_pclk),
    .wea(write_enable),
    .addra(write_addr),
    .dina(write_data),

    .clkb(clk_25mhz),
    .addrb(read_addr),
    .doutb(read_data)
);

// display filter
wire [11:0] filtered_pixel;

image_filter filter (
    .pixel_in(read_data),
    .filter_sel(filter_sel),
    .pixel_out(filtered_pixel)
);

// pclk → clk CDC for detector
wire          det_in_valid;
wire          det_in_frame_start;
wire          det_in_line_start;
wire [7:0]    det_in_pixel;

```

```

pixel_stream_cdc u_cdc (
    .src_clk(camera_pclk),
    .src_rst(reset || !config_done),
    .src_valid(cam_stream_valid),
    .src_frame_start(cam_stream_frame_start),
    .src_line_start(cam_stream_line_start),
    .src_pixel(cam_stream_pixel),

    .dst_clk(clk_25mhz),
    .dst_rst(rst_25),
    .dst_valid(det_in_valid),
    .dst_frame_start(det_in_frame_start),
    .dst_line_start(det_in_line_start),
    .dst_pixel(det_in_pixel)
);

// NN face detector
wire      det_face_valid;
wire [7:0] det_face_x;
wire [6:0] det_face_y;
wire [7:0] det_face_w;
wire [6:0] det_face_h;
wire      det_scan_done;

detector_top #(
    // 500 clears all test negatives while accepting all test positives
    .THRESHOLD(32'sd500),
    // DILATE=3: each patch covers 72x72 pixels in the frame
    .DILATE(3'd3),
    // from weights/scales.vh
    .BBOX_SHIFT(`FC_OUT_SHIFT),
    .CONV1_W_FILE("conv1_w.hex"),
    .CONV1_B_FILE("conv1_b.hex"),
    .CONV2_W_FILE("conv2_w.hex"),
    .CONV2_B_FILE("conv2_b.hex"),
    .CONV3_W_FILE("conv3_w.hex"),
    .CONV3_B_FILE("conv3_b.hex"),
    .FC_W_FILE("fc_w.hex"),
    .FC_B_FILE("fc_b.hex")
) u_detector (
    .clk(clk_25mhz),
    .rst(rst_25),

    .pixel_valid(det_in_valid),
    .frame_start(det_in_frame_start),
    .line_start(det_in_line_start),
    .pixel(det_in_pixel),

    .face_valid(det_face_valid),

```



```

        .face_x(det_face_x),
        .face_y(det_face_y),
        .face_w(det_face_w),
        .face_h(det_face_h),
        .scan_done(det_scan_done)
    );

// CDC face_* from clk → clk_25mhz; toggle-sync, edge-detect, then latch
reg det_update_toggle;
always @(posedge clk_25mhz) begin
    if (rst_25)          det_update_toggle <= 1'b0;
    else if (det_scan_done) det_update_toggle <= ~det_update_toggle;
end

(* ASYNC_REG = "TRUE" *) reg upd_meta;
(* ASYNC_REG = "TRUE" *) reg upd_sync;
reg upd_prev;

reg vga_face_valid;
reg [7:0] vga_face_x;
reg [6:0] vga_face_y;
reg [7:0] vga_face_w;
reg [6:0] vga_face_h;

// hold bbox for a few missed scans to avoid flickering
localparam [3:0] FACE_HOLD_FRAMES = 4'd8;
reg [3:0] face_hold_cnt;

always @(posedge clk_25mhz) begin
    if (rst_25) begin
        upd_meta      <= 1'b0;
        upd_sync      <= 1'b0;
        upd_prev      <= 1'b0;
        vga_face_valid <= 1'b0;
        vga_face_x     <= 8'd0;
        vga_face_y     <= 7'd0;
        vga_face_w     <= 8'd24;
        vga_face_h     <= 7'd24;
        face_hold_cnt  <= 4'd0;
    end else begin
        upd_meta <= det_update_toggle;
        upd_sync <= upd_meta;
        upd_prev <= upd_sync;
        if (upd_sync ^ upd_prev) begin
            if (det_face_valid) begin
                vga_face_valid <= 1'b1;
                vga_face_x     <= det_face_x;
                vga_face_y     <= det_face_y;
                vga_face_w     <= det_face_w;
            end
        end
    end
end

```

```

        vga_face_h      <= det_face_h;
        face_hold_cnt   <= FACE_HOLD_FRAMES;
    end else if (face_hold_cnt != 4'd0) begin
        // no face but still in hold window
        face_hold_cnt   <= face_hold_cnt - 1'b1;
    end else begin
        vga_face_valid <= 1'b0;
    end
end
end
end

// bounding-box overlay
wire [9:0] box_x0 = {vga_face_x, 2'b00};           // 8b * 4 → 10b
wire [9:0] box_y0 = {1'b0, vga_face_y, 2'b00};     // 7b * 4 → 9b, pad to 10b
wire [9:0] box_x1 = box_x0 + {vga_face_w, 2'b00} - 10'd1;
wire [9:0] box_y1 = box_y0 + {1'b0, vga_face_h, 2'b00} - 10'd1;

// registered timing aligns overlay with BRAM latency
wire in_box_x = (vga_x_d >= box_x0) && (vga_x_d <= box_x1);
wire in_box_y = (vga_y_d >= box_y0) && (vga_y_d <= box_y1);

wire on_top    = in_box_x && (vga_y_d >= box_y0) && (vga_y_d <= box_y0 + 10'd1);
wire on_bottom = in_box_x && (vga_y_d >= box_y1 - 10'd1) && (vga_y_d <= box_y1);
wire on_left   = in_box_y && (vga_x_d >= box_x0) && (vga_x_d <= box_x0 + 10'd1);
wire on_right  = in_box_y && (vga_x_d >= box_x1 - 10'd1) && (vga_x_d <= box_x1);

wire on_box = vga_face_valid && (on_top | on_bottom | on_left | on_right);

// VGA pixel output, gated by registered active
assign vga_red    = vga_active_d ? (on_box ? 4'hF : filtered_pixel[11:8]) : 4'h0;
assign vga_green  = vga_active_d ? (on_box ? 4'h0 : filtered_pixel[7:4])  : 4'h0;
assign vga_blue   = vga_active_d ? (on_box ? 4'h0 : filtered_pixel[3:0])  : 4'h0;

// debug LEDs: 0=config_done 1=vsync 2=scan_blink 3=face_latched
//              4=sccb_busy 5=nak_seen 6=frame_ok 7=heartbeat
assign led[0] = config_done;
assign led[1] = camera_vsync;
assign led[2] = det_scan_done;           // blinks once per scan
assign led[3] = det_face_valid;         // face currently latched
assign led[4] = sccb_busy;              // SCCB transaction in progress
assign led[5] = sccb_nak_seen;          // any reg failed after retries
assign led[6] = cap_fok_sync;           // frame format ok
assign led[7] = cap_fhb_sync;           // toggles every frame

endmodule

```

Module Description :

This is the top-level integration module for the FPGA-based real-time face detection system. The design combines OV7670 camera capture, VGA video output, real-time image filtering, clock-domain crossing, and a CNN-based face detector into a complete end-to-end processing pipeline. Captured camera frames are stored in frame buffers, processed by the neural network detector, and displayed on a VGA monitor with real-time bounding-box overlays around detected faces. The module also manages clock generation, camera configuration through SCCB, synchronization between multiple clock domains, and system debugging through onboard LEDs.

Module : ov7670_capture

Module code :

```
`timescale 1ns / 1ps
// Captures raw OV7670 camera data by converting RGB565 to RGB444 for BRAM
// generate a downsampled 160x120 grayscale stream.

module ov7670_capture(
    input wire      pclk,
    input wire      vsync,
    input wire      href,
    input wire [7:0] data_in,
    input wire      reset,

    // RGB444 frame-buffer write port (camera side of dual-port BRAM)
    output reg [16:0] frame_addr,
    output reg [11:0] frame_pixel,
    output reg      frame_we,

    // 160x120 grayscale stream for the NN face detector (pclk domain)
    output reg      stream_valid,
    output reg      stream_frame_start,
    output reg      stream_line_start,
    output reg [7:0] stream_pixel,

    // Diagnostics (pclk domain; top does CDC for LEDs)
    output reg      frame_format_ok,
    output reg      frame_heartbeat
);

    // ----- Edge detectors for vsync/href -----
    reg prev_vsync, prev_href;

    // ----- Byte pairing & native pixel counters -----
    reg [7:0] hi_byte;
    reg      byte_phase;          // 0 = MSB, 1 = LSB
    reg [10:0] cam_col;           // 0..639
    reg [9:0] cam_row;            // 0..479

    // RGB565 -> RGB444 decode
    wire [3:0] px_r = hi_byte[7:4];
    wire [3:0] px_g = { hi_byte[2:0], data_in[7] };
    wire [3:0] px_b = data_in[4:1];

    // luma proxy for the detector: (R+G+B) scaled to ~8 bits.
    // sum range 0..45 -> sum * 5 ≈ 0..225 (fits in 8 bits, no clamp needed).
    wire [5:0] chan_sum = {2'b00, px_r} + {2'b00, px_g} + {2'b00, px_b};
    wire [7:0] gray8    = {chan_sum, 2'b00} + {2'b00, chan_sum}; // sum*5
```

```

// 2x2 store -> 320x240 for BRAM
wire keep_for_buffer = (cam_col[0] == 1'b0) && (cam_row[0] == 1'b0);
// 4x4 store -> 160x120 for Grayscale stream
wire keep_for_stream = (cam_col[1:0] == 2'b00) && (cam_row[1:0] == 2'b00);

localparam integer MAX_ADDR = 320 * 240 - 1;    // 76799

// -----
// Main capture FSM
// -----
always @(posedge pclk or posedge reset) begin
    if (reset) begin
        frame_addr        <= 17'd0;
        frame_pixel       <= 12'h000;
        frame_we          <= 1'b0;
        stream_valid      <= 1'b0;
        stream_frame_start <= 1'b0;
        stream_line_start <= 1'b0;
        stream_pixel      <= 8'd0;
        byte_phase        <= 1'b0;
        cam_col            <= 11'd0;
        cam_row            <= 10'd0;
        prev_vsync        <= 1'b0;
        prev_href          <= 1'b0;
        hi_byte            <= 8'd0;
    end else begin
        prev_vsync        <= vsync;
        prev_href          <= href;
        frame_we          <= 1'b0;
        stream_valid      <= 1'b0;
        stream_frame_start <= 1'b0;
        stream_line_start <= 1'b0;

        // Address Increment
        if (frame_we && frame_addr < MAX_ADDR[16:0]) begin
            frame_addr <= frame_addr + 1'b1;
        end

        // Rising edge of VSYNC = reset for the next frame.
        if (!prev_vsync && vsync) begin
            frame_addr <= 17'd0;
            cam_col    <= 11'd0;
            cam_row    <= 10'd0;
            byte_phase <= 1'b0;
        end else if (href) begin
            // store pair of byte
            if (byte_phase == 1'b0) begin
                hi_byte    <= data_in;
                byte_phase <= 1'b1;
            end
        end
    end
end

```

```

        end else begin
            byte_phase <= 1'b0;

            // data to write to BRAM
            if (keep_for_buffer && frame_addr <= MAX_ADDR[16:0]) begin
                frame_pixel <= {px_r, px_g, px_b};
                frame_we      <= 1'b1;
            end

            // data for stream (NN face detection & Grayscale)
            if (keep_for_stream) begin
                stream_valid      <= 1'b1;
                stream_pixel      <= gray8;
                stream_frame_start <= (cam_col == 11'd0) && (cam_row == 10'd0);
                stream_line_start <= (cam_col == 11'd0);
            end

            cam_col <= cam_col + 11'd1;
        end
    end else begin
        // end line or blanking
        byte_phase <= 1'b0;
        if (prev_href && !href) begin
            cam_col <= 11'd0;
            cam_row <= cam_row + 10'd1;
        end
    end
end

// sanity check (1280 pclk/line & 480 lines/frame)
reg [11:0] pclks_this_line;
reg [9:0]  lines_this_frame;
reg       line_ok_sticky;
reg       prev_href_d;
reg       prev_vsync_d;

always @(posedge pclk or posedge reset) begin
    if (reset) begin
        pclks_this_line <= 12'd0;
        lines_this_frame <= 10'd0;
        line_ok_sticky  <= 1'b1;
        prev_href_d     <= 1'b0;
        prev_vsync_d    <= 1'b0;
        frame_format_ok <= 1'b0;
        frame_heartbeat <= 1'b0;
    end else begin
        prev_href_d <= href;
        prev_vsync_d <= vsync;
    end
end

```

```

        if (href) pclk_this_line <= pclk_this_line + 1'b1;

        if (prev_href_d && !href) begin
            if (pclk_this_line != 12'd1280) line_ok_sticky <= 1'b0;
            pclk_this_line <= 12'd0;
            lines_this_frame <= lines_this_frame + 1'b1;
        end

        if (!prev_vsync_d && vsync) begin
            frame_format_ok <= line_ok_sticky && (lines_this_frame == 10'd480);
            lines_this_frame <= 10'd0;
            line_ok_sticky <= 1'b1;
            frame_heartbeat <= ~frame_heartbeat;
        end
    end
end

endmodule

```

Module Description :

This module is responsible for interfacing with the OV7670 camera sensor and capturing raw image data in real time. The module converts incoming RGB565 pixel data into RGB444 format for frame buffer storage and simultaneously generates a downsampled grayscale stream for neural network-based face detection.

Module : ov7670_config

Module code :

```
`timescale 1ns / 1ps
// Initializes the camera for VGA 640x480 RGB565 output
// running through a 36-register sequence via open-loop SCCB/I2C.

module ov7670_config(
    input  wire clk,           // 100 MHz system clock
    input  wire reset,        // async, active-high
    output reg  sioc,          // SCCB clock line
    inout  wire siod,          // SCCB data line (open-drain)
    output wire config_done,
    output wire sccb_busy,
    output wire sccb_nak_seen
);

    // Clock divider: 100MHz -> 400kHz tick (~2.5us per phase).
    // This gives us a ~100kHz SCL, well within the SCCB spec.
    parameter [8:0] PHASE_TICKS = 9'd250;

    reg [8:0] tick_div;
    reg      phase_tick;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            tick_div  <= 9'd0;
            phase_tick <= 1'b0;
        end else if (tick_div == PHASE_TICKS - 1'b1) begin
            tick_div  <= 9'd0;
            phase_tick <= 1'b1;
        end else begin
            tick_div  <= tick_div + 1'b1;
            phase_tick <= 1'b0;
        end
    end

    // Register table - 36 entries
    localparam integer NUM_CMDS = 36;
    localparam [7:0] CAM_WADDR = 8'h42; // OV7670 slave write address

    reg [7:0] cmd_index;
    reg [15:0] cmd_word;

    always @(*) begin
        case (cmd_index)
            8'd0: cmd_word = 16'h12_80; // COM7 - software reset
            8'd1: cmd_word = 16'h11_01; // CLKRC - enable internal prescaler
            8'd2: cmd_word = 16'h6B_4A; // DBLV - PLL stabilization
        end
    end
end
```



```

8'd3: cmd_word = 16'h12_04; // COM7 - select RGB output
8'd4: cmd_word = 16'h8C_00; // RGB444 disable -> use RGB565
8'd5: cmd_word = 16'h40_D0; // COM15 - RGB565, full output range
8'd6: cmd_word = 16'h3A_04; // TSLB
8'd7: cmd_word = 16'hB0_84; // (reserved register required for clean RGB)
8'd8 : cmd_word = 16'h3D_C0; //UV saturation
8'd9 : cmd_word = 16'h01_C0; // blue gain
8'd10 : cmd_word = 16'h02_F0; // red gain
8'd11 : cmd_word = 16'h13_E5; // COM8 - auto control

// --- Mild saturation boost ---
8'd12 : cmd_word = 16'h4F_75;
8'd13 : cmd_word = 16'h50_75;
8'd14 : cmd_word = 16'h51_00;
8'd15 : cmd_word = 16'h52_22;
8'd16 : cmd_word = 16'h53_5E;
8'd17 : cmd_word = 16'h54_80;
8'd18 : cmd_word = 16'h58_9E;

// --- Gamma / contrast ---
8'd19 : cmd_word = 16'h7A_20;
8'd20 : cmd_word = 16'h7B_10;
8'd21 : cmd_word = 16'h7C_1E;
8'd22 : cmd_word = 16'h7D_35;
8'd23 : cmd_word = 16'h7E_5A;
8'd24 : cmd_word = 16'h7F_69;
8'd25 : cmd_word = 16'h80_76;
8'd26 : cmd_word = 16'h81_80;
8'd27 : cmd_word = 16'h82_88;
8'd28 : cmd_word = 16'h83_8F;
8'd29 : cmd_word = 16'h84_96;
8'd30 : cmd_word = 16'h85_A3;
8'd31 : cmd_word = 16'h86_AF;
8'd32 : cmd_word = 16'h87_C4;
8'd33 : cmd_word = 16'h88_D7;
8'd34 : cmd_word = 16'h89_E8;

// --- Slight contrast boost ---
8'd35 : cmd_word = 16'h56_40;
default: cmd_word = 16'h0000;
endcase
end

// SCCB FSM: 4-phase bit timing (SCL low = drive, SCL high = hold)
// Send 3 bytes (Slave/ Reg/ Data)
localparam [3:0] ST_IDLE      = 4'd0;
localparam [3:0] ST_START    = 4'd1;
localparam [3:0] ST_SCL_LO_A = 4'd2;
localparam [3:0] ST_DRIVE_DATA = 4'd3;

```

```

localparam [3:0] ST_SCL_HI_A    = 4'd4;
localparam [3:0] ST_HOLD_A      = 4'd5;
localparam [3:0] ST_SCL_LO_B    = 4'd6;
localparam [3:0] ST_RELEASE     = 4'd7;
localparam [3:0] ST_SCL_HI_B    = 4'd8;
localparam [3:0] ST_HOLD_B      = 4'd9;
localparam [3:0] ST_STOP_LO     = 4'd10;
localparam [3:0] ST_STOP_DRV    = 4'd11;
localparam [3:0] ST_STOP_HI     = 4'd12;
localparam [3:0] ST_NEXT_LO     = 4'd13;
localparam [3:0] ST_NEXT_DRV    = 4'd14;
localparam [3:0] ST_FINISH      = 4'd15;

reg [3:0] fsm_state;
reg [5:0] bit_idx;              // 23..0 across the 3-byte burst
reg [23:0] tx_shift;
reg      sda_q;
reg      sda_oe;
reg [10:0] post_reset_cnt;      // ~1 ms idle after soft reset

// SCCB / open-drain: pull SDA low when we want a 0, otherwise Hi-Z.
assign siod = (sda_oe && sda_q == 1'b0) ? 1'b0 : 1'bz;

assign sccb_busy      = (fsm_state != ST_IDLE);
assign sccb_nak_seen = 1'b0;
assign config_done    = (cmd_index >= NUM_CMDS) && (fsm_state == ST_IDLE);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        fsm_state      <= ST_IDLE;
        bit_idx        <= 6'd0;
        tx_shift       <= 24'd0;
        cmd_index      <= 8'd0;
        sioc           <= 1'b1;
        sda_q          <= 1'b1;
        sda_oe         <= 1'b1;
        post_reset_cnt <= 11'd0;
    end else if (phase_tick) begin
        case (fsm_state)
            ST_IDLE: begin
                if (cmd_index < NUM_CMDS) begin
                    // Wait ~1 ms after the soft reset command before going on
                    if (cmd_index == 8'd1 && post_reset_cnt < 11'd400) begin
                        post_reset_cnt <= post_reset_cnt + 1'b1;
                    end else begin
                        tx_shift <= {CAM_WADDR, cmd_word[15:8], cmd_word[7:0]};
                        sda_oe   <= 1'b1;
                        sioc     <= 1'b1;
                        sda_q    <= 1'b1;
                    end
                end
            end
        end case
    end
end

```

```

        fsm_state <= ST_START;
    end
end
end

// START: SDA falls while SCL is high
ST_START: begin
    sda_q      <= 1'b0;
    bit_idx    <= 6'd23;
    fsm_state <= ST_SCL_LO_A;
end

// Send-bit phases
// down SCL for update data
ST_SCL_LO_A:  begin sioc <= 1'b0;          fsm_state <= ST_DRIVE_DATA; end
ST_DRIVE_DATA: begin sda_q <= tx_shift[bit_idx]; fsm_state <= ST_SCL_HI_A; end
// rising SCL (flag ready data)
ST_SCL_HI_A:  begin sioc <= 1'b1;          fsm_state <= ST_HOLD_A;      end
ST_HOLD_A: begin
    if (bit_idx == 6'd16 || bit_idx == 6'd8 || bit_idx == 6'd0) begin
        fsm_state <= ST_SCL_LO_B;          // last data bit of a byte -> ACK phase
    end else begin
        bit_idx    <= bit_idx - 1'b1;
        fsm_state <= ST_SCL_LO_A;
    end
end
end

// Don't-care / Dummy ACK phases (SDA tristated)
ST_SCL_LO_B: begin sioc <= 1'b0; fsm_state <= ST_RELEASE; end
ST_RELEASE:  begin sda_oe <= 1'b0; fsm_state <= ST_SCL_HI_B; end
ST_SCL_HI_B: begin sioc <= 1'b1; fsm_state <= ST_HOLD_B; end
ST_HOLD_B: begin
    if (bit_idx == 6'd0)
        fsm_state <= ST_STOP_LO;          // burst complete, STOP next
    else begin
        bit_idx    <= bit_idx - 1'b1;
        fsm_state <= ST_NEXT_LO;          // continue to next byte
    end
end
end

ST_NEXT_LO:  begin sioc <= 1'b0; fsm_state <= ST_NEXT_DRV; end
ST_NEXT_DRV: begin sda_oe <= 1'b1; fsm_state <= ST_DRIVE_DATA; end

// STOP: SDA rises while SCL is high
ST_STOP_LO:  begin sioc <= 1'b0; fsm_state <= ST_STOP_DRV; end
ST_STOP_DRV: begin sda_oe <= 1'b1; sda_q <= 1'b0; fsm_state <= ST_STOP_HI; end
ST_STOP_HI:  begin sioc <= 1'b1; fsm_state <= ST_FINISH; end
ST_FINISH: begin
    sda_q      <= 1'b1;
    cmd_index <= cmd_index + 1'b1;
end

```

```
        fsm_state <= ST_IDLE;
    end

    default: fsm_state <= ST_IDLE;
endcase
end
end

endmodule
```

Module Description :

This module initializes the OV7670 camera sensor through the SCCB communication interface. The module sequentially transmits camera register settings required for VGA 640×480 RGB565 image output.

Module : pixel_stream_cdc

Module code :

```
`timescale 1ns / 1ps
`default_nettype none
// Performs safe Clock Domain Crossing (CDC) for pixel data
// using toggle-based synchronizer, ensuring data stability when moving from the camera
pclk

module pixel_stream_cdc(
    // Source domain: camera pclk
    input wire      src_clk,
    input wire      src_rst,
    input wire      src_valid,
    input wire      src_line_start,
    input wire [7:0] src_pixel,

    // Destination domain: system clk
    input wire      dst_clk,
    input wire      dst_rst,
    output reg      dst_valid,
    output reg      dst_frame_start,
    output reg      dst_line_start,
    output reg [7:0] dst_pixel
);

    //flag every new pixel
    reg      src_toggle;
    //latched
    reg [7:0] src_pixel_l;
    reg      src_frame_start_l;
    reg      src_line_start_l;

    always @(posedge src_clk or posedge src_rst) begin
        if (src_rst) begin
            src_toggle      <= 1'b0;
            src_pixel_l      <= 8'd0;
            src_frame_start_l <= 1'b0;
            src_line_start_l <= 1'b0;
        end else if (src_valid) begin
            src_pixel_l      <= src_pixel;
            src_frame_start_l <= src_frame_start;
            src_line_start_l <= src_line_start;
            src_toggle      <= ~src_toggle;
        end
    end

    // 2-stage synchronizer (FF) for stability
    (* ASYNC_REG = "TRUE" *) reg toggle_meta;
```

```

(* ASYNC_REG = "TRUE" *) reg toggle_sync;
// Edge-detection
reg toggle_prev;

always @(posedge dst_clk or posedge dst_rst) begin
    if (dst_rst) begin
        toggle_meta    <= 1'b0;
        toggle_sync    <= 1'b0;
        toggle_prev    <= 1'b0;
        dst_valid       <= 1'b0;
        dst_frame_start <= 1'b0;
        dst_line_start  <= 1'b0;
        dst_pixel       <= 8'd0;
    end else begin
        toggle_meta <= src_toggle;
        toggle_sync <= toggle_meta;
        toggle_prev <= toggle_sync;

        // One-cycle pulse on each toggle edge.
        dst_valid <= (toggle_sync ^ toggle_prev);
        if (toggle_sync ^ toggle_prev) begin
            dst_pixel    <= src_pixel_l;
            dst_frame_start <= src_frame_start_l;
            dst_line_start <= src_line_start_l;
        end
    end
end

endmodule

`default_nettype wire

```

Module Description :

This module safely transfers grayscale pixel stream data from the camera pixel clock domain (src_clk) to the system processing clock domain (dst_clk). The module prevents metastability and ensures reliable synchronization between asynchronous clock domains.

Module : vga_controller

Module code :

```
`timescale 1ns / 1ps
// Generates VGA 640x480@60Hz timing signals by tracking pixel coordinates,
// maintaining proper horizontal/vertical sync pulses and active display regions.

module vga_controller(
    input wire clk,          // 25 MHz pixel clock from clk_wiz_0
    input wire reset,

    output reg hsync,
    output reg vsync,
    output wire active,      // High during active display region
    output wire [9:0] x_pos, // Current X position (0-799)
    output wire [9:0] y_pos  // Current Y position (0-524)
);

    // VGA 640x480 @ 60Hz timing parameters
    // Horizontal timing (pixels)
    parameter H_DISPLAY = 640;
    parameter H_FRONT   = 16;
    parameter H_SYNC    = 96;
    parameter H_BACK    = 48;
    parameter H_TOTAL   = H_DISPLAY + H_FRONT + H_SYNC + H_BACK; // 800

    // Vertical timing (lines)
    parameter V_DISPLAY = 480;
    parameter V_FRONT   = 10;
    parameter V_SYNC    = 2;
    parameter V_BACK    = 33;
    parameter V_TOTAL   = V_DISPLAY + V_FRONT + V_SYNC + V_BACK; // 525

    // Current position counters
    reg [9:0] h_count;
    reg [9:0] v_count;

    assign x_pos = h_count;
    assign y_pos = v_count;

    // Active display region
    assign active = (h_count < H_DISPLAY) && (v_count < V_DISPLAY);

    // Horizontal counter
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            h_count <= 0;
        end else begin
            if (h_count >= H_TOTAL - 1) begin
```

```

        h_count <= 0;
    end else begin
        h_count <= h_count + 1;
    end
end

end

end

// Vertical counter
always @(posedge clk or posedge reset) begin
    if (reset) begin
        v_count <= 0;
    end else begin
        if (h_count >= H_TOTAL - 1) begin
            if (v_count >= V_TOTAL - 1) begin
                v_count <= 0;
            end else begin
                v_count <= v_count + 1;
            end
        end
    end
end

end

end

// Generate HSYNC
always @(posedge clk or posedge reset) begin
    if (reset) begin
        hsync <= 1;
    end else begin
        //sync period
        if (h_count >= (H_DISPLAY + H_FRONT) &&
            h_count < (H_DISPLAY + H_FRONT + H_SYNC)) begin
            hsync <= 0;
        end else begin
            hsync <= 1;
        end
    end
end

end

// Generate VSYNC
always @(posedge clk or posedge reset) begin
    if (reset) begin
        vsync <= 1;
    end else begin
        //sync period
        if (v_count >= (V_DISPLAY + V_FRONT) &&
            v_count < (V_DISPLAY + V_FRONT + V_SYNC)) begin
            vsync <= 0;
        end else begin
            vsync <= 1;
        end
    end
end

```



```
end  
end  
  
endmodule
```

Module Description :

This module generates VGA timing signals for 640×480 resolution at 60 Hz using a 25 MHz pixel clock. Horizontal and vertical counters are used to track the current pixel position and produce the required HSYNC and VSYNC synchronization pulses according to the VGA standard. The module also provides active display region detection and real-time pixel coordinate outputs for video rendering and overlay processing.

Module : filter_frame_buffer

Module code :

```
`timescale 1ns / 1ps
// Frame Buffer Module (Dual-Port Block RAM)

module filter_frame_buffer(
    // Port A (Write - Camera side)
    input wire clka,
    input wire wea,
    input wire [16:0] addra,    // 17 bits for 76800 addresses
    input wire [11:0] dina,

    // Port B (Read - VGA side)
    input wire clk_b,
    input wire [16:0] addrb,
    output reg [11:0] doutb
);

    // Block RAM storage
    (* ram_style = "block" *) reg [11:0] ram [0:76799];

    // Initialize RAM to black
    integer i;
    initial begin
        for (i = 0; i < 76800; i = i + 1) begin
            ram[i] = 12'h000;
        end
    end

    // Port A: Write operation
    always @(posedge clka) begin
        if (wea) begin
            ram[addra] <= dina;
        end
    end

    // Port B: Read operation
    always @(posedge clk_b) begin
        doutb <= ram[addrb];
    end

endmodule
```

Module Description :

This module implements a dual-port frame buffer using FPGA Block RAM to store RGB444 image data captured from the OV7670 camera. The write port operates in the camera clock domain, continuously storing incoming pixels, while the read port

operates in the VGA clock domain to support simultaneous real-time display. The dual-port architecture allows concurrent camera capture and VGA rendering without memory access conflicts.

Module : image_filter

Module code :

```
`timescale 1ns / 1ps

// Implements image processing filters:
// 000: No filter (original image)
// 001: Grayscale conversion
// 010: Color inversion (negative)
// 011: Threshold (binary black/white)
// 100: Red channel only
// 101: Green channel only
// 110: Blue channel only
// 111: Brightness boost

module image_filter(
    input wire [11:0] pixel_in,    // RGB444 input
    input wire [2:0] filter_sel,   // Filter selection
    output reg [11:0] pixel_out    // RGB444 output
);

    // Extract color channels
    wire [3:0] red_in  = pixel_in[11:8];
    wire [3:0] green_in = pixel_in[7:4];
    wire [3:0] blue_in  = pixel_in[3:0];

    // Intermediate values
    reg [3:0] red_out;
    reg [3:0] green_out;
    reg [3:0] blue_out;

    // Grayscale calculation (weighted average)
    //  $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ 
    // Approximation:  $Y = (R + 2 \cdot G + B) / 4$ 
    wire [5:0] gray_sum = {2'b00, red_in} + {1'b0, green_in, 1'b0} + {2'b00, blue_in};
    wire [3:0] gray = gray_sum[5:2]; // Divide by 4

    // Threshold calculation (for binary image)
    wire [5:0] brightness = {2'b00, red_in} + {2'b00, green_in} + {2'b00, blue_in};
    wire is_bright = (brightness > 6'd24); // Threshold at mid-level

    always @(*) begin
        case (filter_sel)
            3'b000: begin
                // No filter - pass through
                red_out = red_in;
                green_out = green_in;
                blue_out = blue_in;
            end
        end
    end
```

```
3'b001: begin
    // Grayscale filter
    red_out = gray;
    green_out = gray;
    blue_out = gray;
end

3'b010: begin
    // Color inversion (negative)
    red_out = ~red_in;
    green_out = ~green_in;
    blue_out = ~blue_in;
end

3'b011: begin
    // Threshold (binary black/white)
    if (is_bright) begin
        red_out = 4'hF;
        green_out = 4'hF;
        blue_out = 4'hF;
    end else begin
        red_out = 4'h0;
        green_out = 4'h0;
        blue_out = 4'h0;
    end
end

3'b100: begin
    // Red channel only
    red_out = red_in;
    green_out = 4'h0;
    blue_out = 4'h0;
end

3'b101: begin
    // Green channel only
    red_out = 4'h0;
    green_out = green_in;
    blue_out = 4'h0;
end

3'b110: begin
    // Blue channel only
    red_out = 4'h0;
    green_out = 4'h0;
    blue_out = blue_in;
end
```

```

3'b111: begin
    // Brightness boost (saturate at max)
    red_out = (red_in >= 4'hC) ? 4'hF : red_in + 4'h3;
    green_out = (green_in >= 4'hC) ? 4'hF : green_in + 4'h3;
    blue_out = (blue_in >= 4'hC) ? 4'hF : blue_in + 4'h3;
end

default: begin
    red_out = red_in;
    green_out = green_in;
    blue_out = blue_in;
end
endcase

pixel_out = {red_out, green_out, blue_out};
end

endmodule

```

Module Description :

This module performs real-time image filtering on RGB444 pixel data before VGA display output. Multiple selectable filters are implemented, including grayscale conversion, color inversion, binary thresholding, individual RGB channel extraction, and brightness enhancement. The design uses purely combinational logic for single-cycle processing, enabling low-latency image manipulation suitable for real-time FPGA video pipelines.

Module : detector_top

Module code :

```
// sliding-window face detector
// scans one full frame, picks the highest-confidence patch and outputs dequantized
// face_* coords

`include "scales.vh"
`default_nettype none

module detector_top #(
    parameter FRAME_W      = 160,
    parameter FRAME_H      = 120,
    parameter PATCH        = 24,
    parameter STRIDE        = 16,
    parameter [2:0] DILATE = 3'd3,
    // right-shift to dequantize fc bbox outputs into patch pixels (from scales.vh)
    parameter integer BBOX_SHIFT = `FC_OUT_SHIFT,
    // raw int32 conf threshold; increase for stricter detection
    parameter signed [31:0] THRESHOLD = 32'sd0,

    parameter CONV1_W_FILE = "weights/conv1_w.hex",
    parameter CONV1_B_FILE = "weights/conv1_b.hex",
    parameter CONV2_W_FILE = "weights/conv2_w.hex",
    parameter CONV2_B_FILE = "weights/conv2_b.hex",
    parameter CONV3_W_FILE = "weights/conv3_w.hex",
    parameter CONV3_B_FILE = "weights/conv3_b.hex",
    parameter FC_W_FILE    = "weights/fc_w.hex",
    parameter FC_B_FILE    = "weights/fc_b.hex"
) (
    input  wire      clk,
    input  wire      rst,

    input  wire      pixel_valid,
    input  wire      frame_start,
    input  wire      line_start,
    input  wire [7:0] pixel,

    output reg       face_valid,
    output reg [7:0] face_x,
    output reg [6:0] face_y,
    output reg [7:0] face_w,
    output reg [6:0] face_h,

    output reg       scan_done
);

localparam integer EFF_PATCH = PATCH * DILATE;
```

```

// FSM states
localparam [3:0] S_WAIT      = 4'd0;
localparam [3:0] S_INGEST    = 4'd1;
localparam [3:0] S_INIT_SCAN = 4'd2;
localparam [3:0] S_PE_START  = 4'd3;
localparam [3:0] S_PE_WAIT   = 4'd4;
localparam [3:0] S_C1_START  = 4'd5;
localparam [3:0] S_C1_WAIT   = 4'd6;
localparam [3:0] S_C2_START  = 4'd7;
localparam [3:0] S_C2_WAIT   = 4'd8;
localparam [3:0] S_C3_START  = 4'd9;
localparam [3:0] S_C3_WAIT   = 4'd10;
localparam [3:0] S_FC_START  = 4'd11;
localparam [3:0] S_FC_WAIT   = 4'd12;
localparam [3:0] S_SCORE     = 4'd13;
localparam [3:0] S_LATCH     = 4'd14;

reg [3:0] state, next;

reg [7:0] scan_x;
reg [6:0] scan_y;

// best detection so far
reg signed [31:0] best_conf;
reg              best_valid;
reg [7:0]        best_face_x;
reg [6:0]        best_face_y;
reg [7:0]        best_face_w;
reg [6:0]        best_face_h;

// sub-module wires
wire          fb_frame_done;
wire [14:0]    fb_r_addr;
wire [7:0]     fb_r_data;

reg          pe_start;
wire         pe_done;
wire         pe_ab_we;
wire [9:0]    pe_ab_w_addr;
wire signed [7:0] pe_ab_w_data;

wire [9:0]    inab_r_addr;
wire signed [7:0] inab_r_data;

wire [6:0]    c1_w_addr;
wire signed [7:0] c1_w_data;
wire [2:0]    c1_b_addr;
wire signed [31:0] c1_b_data;
reg          c1_start;

```



```

wire          c1_done;
wire          c1_out_we;
wire [9:0]    c1_out_w_addr;
wire signed [7:0] c1_out_w_data;

wire [9:0]    clab_r_addr;
wire signed [7:0] clab_r_data;

wire [10:0]    c2_w_addr;
wire signed [7:0] c2_w_data;
wire [3:0]    c2_b_addr;
wire signed [31:0] c2_b_data;
reg          c2_start;
wire          c2_done;
wire          c2_out_we;
wire [8:0]    c2_out_w_addr;
wire signed [7:0] c2_out_w_data;

wire [8:0]    c2ab_r_addr;
wire signed [7:0] c2ab_r_data;

wire [11:0]    c3_w_addr;
wire signed [7:0] c3_w_data;
wire [3:0]    c3_b_addr;
wire signed [31:0] c3_b_data;
reg          c3_start;
wire          c3_done;
wire          c3_out_we;
wire [7:0]    c3_out_w_addr;
wire signed [7:0] c3_out_w_data;

wire [7:0]    c3ab_r_addr;
wire signed [7:0] c3ab_r_data;

// 720 weights + 5 biases for 5 fc outputs
wire [9:0]    fc_w_addr;
wire signed [7:0] fc_w_data;
wire [2:0]    fc_b_addr;
wire signed [31:0] fc_b_data;

reg          fc_start;
wire          fc_done;
wire signed [31:0] fc_conf;
wire signed [31:0] fc_x0;
wire signed [31:0] fc_y0;
wire signed [31:0] fc_w_raw;
wire signed [31:0] fc_h_raw;

// sub-module instances

```

```

frame_buffer u_fb (
    .clk(clk), .rst(rst),
    .pixel_valid(pixel_valid), .frame_start(frame_start), .line_start(line_start),
    .pixel(pixel), .frame_done(fb_frame_done),
    .r_addr(fb_r_addr), .r_data(fb_r_data)
);

patch_extractor #(.FRAME_W(FRAME_W), .PATCH(PATCH)) u_pe (
    .clk(clk), .rst(rst),
    .start(pe_start), .patch_x(scan_x), .patch_y(scan_y),
    .dilate_in(DILATE), .done(pe_done),
    .fb_r_addr(fb_r_addr), .fb_r_data(fb_r_data),
    .ab_we(pe_ab_we), .ab_w_addr(pe_ab_w_addr), .ab_w_data(pe_ab_w_data)
);

act_buffer #(.WIDTH(8), .DEPTH(576), .ADDR_W(10)) u_inab (
    .clk(clk),
    .we(pe_ab_we), .w_addr(pe_ab_w_addr), .w_data(pe_ab_w_data),
    .r_addr(inab_r_addr), .r_data(inab_r_data)
);

weight_rom #(.WIDTH(8), .DEPTH(72), .ADDR_W(7), .MEM_FILE(CONV1_W_FILE))
    u_c1_w (.clk(clk), .addr(c1_w_addr), .data(c1_w_data));
weight_rom #(.WIDTH(32), .DEPTH(8), .ADDR_W(3), .MEM_FILE(CONV1_B_FILE))
    u_c1_b (.clk(clk), .addr(c1_b_addr), .data(c1_b_data));

conv_layer #(.IN_CH(1), .OUT_CH(8), .IN_H(24), .IN_W(24),
    .K(3), .STRIDE(2), .REQUANT_SHIFT(`CONV1_SHIFT)) u_c1 (
    .clk(clk), .rst(rst), .start(c1_start), .done(c1_done),
    .in_addr(inab_r_addr), .in_data(inab_r_data),
    .w_addr(c1_w_addr), .w_data(c1_w_data),
    .b_addr(c1_b_addr), .b_data(c1_b_data),
    .out_we(c1_out_we), .out_addr(c1_out_w_addr), .out_data(c1_out_w_data)
);

act_buffer #(.WIDTH(8), .DEPTH(968), .ADDR_W(10)) u_clab (
    .clk(clk),
    .we(c1_out_we), .w_addr(c1_out_w_addr), .w_data(c1_out_w_data),
    .r_addr(clab_r_addr), .r_data(clab_r_data)
);

weight_rom #(.WIDTH(8), .DEPTH(1152), .ADDR_W(11), .MEM_FILE(CONV2_W_FILE))
    u_c2_w (.clk(clk), .addr(c2_w_addr), .data(c2_w_data));
weight_rom #(.WIDTH(32), .DEPTH(16), .ADDR_W(4), .MEM_FILE(CONV2_B_FILE))
    u_c2_b (.clk(clk), .addr(c2_b_addr), .data(c2_b_data));

conv_layer #(.IN_CH(8), .OUT_CH(16), .IN_H(11), .IN_W(11),
    .K(3), .STRIDE(2), .REQUANT_SHIFT(`CONV2_SHIFT)) u_c2 (
    .clk(clk), .rst(rst), .start(c2_start), .done(c2_done),

```

```

        .in_addr(c1ab_r_addr), .in_data(c1ab_r_data),
        .w_addr(c2_w_addr),    .w_data(c2_w_data),
        .b_addr(c2_b_addr),    .b_data(c2_b_data),
        .out_we(c2_out_we), .out_addr(c2_out_w_addr), .out_data(c2_out_w_data)
    );

    act_buffer #(.WIDTH(8), .DEPTH(400), .ADDR_W(9)) u_c2ab (
        .clk(clk),
        .we(c2_out_we), .w_addr(c2_out_w_addr), .w_data(c2_out_w_data),
        .r_addr(c2ab_r_addr), .r_data(c2ab_r_data)
    );

    weight_rom #(.WIDTH(8), .DEPTH(2304), .ADDR_W(12), .MEM_FILE(CONV3_W_FILE))
        u_c3_w (.clk(clk), .addr(c3_w_addr), .data(c3_w_data));
    weight_rom #(.WIDTH(32), .DEPTH(16), .ADDR_W(4), .MEM_FILE(CONV3_B_FILE))
        u_c3_b (.clk(clk), .addr(c3_b_addr), .data(c3_b_data));

    conv_layer #(.IN_CH(16), .OUT_CH(16), .IN_H(5), .IN_W(5),
        .K(3), .STRIDE(1), .REQUANT_SHIFT(`CONV3_SHIFT)) u_c3 (
        .clk(clk), .rst(rst), .start(c3_start), .done(c3_done),
        .in_addr(c2ab_r_addr), .in_data(c2ab_r_data),
        .w_addr(c3_w_addr),    .w_data(c3_w_data),
        .b_addr(c3_b_addr),    .b_data(c3_b_data),
        .out_we(c3_out_we), .out_addr(c3_out_w_addr), .out_data(c3_out_w_data)
    );

    act_buffer #(.WIDTH(8), .DEPTH(144), .ADDR_W(8)) u_c3ab (
        .clk(clk),
        .we(c3_out_we), .w_addr(c3_out_w_addr), .w_data(c3_out_w_data),
        .r_addr(c3ab_r_addr), .r_data(c3ab_r_data)
    );

    // FC ROMs
    weight_rom #(.WIDTH(8), .DEPTH(720), .ADDR_W(10), .MEM_FILE(FC_W_FILE))
        u_fc_w (.clk(clk), .addr(fc_w_addr), .data(fc_w_data));
    weight_rom #(.WIDTH(32), .DEPTH(5), .ADDR_W(3), .MEM_FILE(FC_B_FILE))
        u_fc_b (.clk(clk), .addr(fc_b_addr), .data(fc_b_data));

    fc_layer #(.IN_LEN(144), .OUT_LEN(5)) u_fc (
        .clk(clk), .rst(rst), .start(fc_start), .done(fc_done),
        .in_addr(c3ab_r_addr), .in_data(c3ab_r_data),
        .w_addr(fc_w_addr),    .w_data(fc_w_data),
        .b_addr(fc_b_addr),    .b_data(fc_b_data),
        .out_conf(fc_conf),
        .out_x0 (fc_x0),
        .out_y0 (fc_y0),
        .out_w  (fc_w_raw),
        .out_h  (fc_h_raw)
    );

```

```

// dequantize bbox outputs to patch pixels
wire signed [31:0] x0_shifted = fc_x0    >>> BBOX_SHIFT;
wire signed [31:0] y0_shifted = fc_y0    >>> BBOX_SHIFT;
wire signed [31:0] w_shifted  = fc_w_raw >>> BBOX_SHIFT;
wire signed [31:0] h_shifted  = fc_h_raw >>> BBOX_SHIFT;

function [4:0] clamp_xy;
    input signed [31:0] v;
    begin
        if      (v <= 32'sd0)  clamp_xy = 5'd0;
        else if (v >= 32'sd23) clamp_xy = 5'd23;
        else                clamp_xy = v[4:0];
    end
endfunction

function [5:0] clamp_wh;
    input signed [31:0] v;
    begin
        if      (v <= 32'sd1)  clamp_wh = 6'd1;
        else if (v >= 32'sd24) clamp_wh = 6'd24;
        else                clamp_wh = v[5:0];
    end
endfunction

wire [4:0] dx_patch = clamp_xy(x0_shifted);
wire [4:0] dy_patch = clamp_xy(y0_shifted);
wire [5:0] dw_patch = clamp_wh(w_shifted);
wire [5:0] dh_patch = clamp_wh(h_shifted);

// scale patch coords to frame coords via DILATE then offset by scan corner
wire [7:0] dx_frame = dx_patch * DILATE;
wire [7:0] dy_frame = dy_patch * DILATE;
wire [7:0] dw_frame = dw_patch * DILATE;
wire [7:0] dh_frame = dh_patch * DILATE;

wire [7:0] frame_face_x = scan_x + dx_frame;
wire [6:0] frame_face_y = scan_y + dy_frame[6:0];
wire [7:0] frame_face_w = dw_frame;
wire [6:0] frame_face_h = dh_frame[6:0];

// scan extents
wire scan_x_last = (scan_x + STRIDE > FRAME_W - EFF_PATCH);
wire scan_y_last = (scan_y + STRIDE > FRAME_H - EFF_PATCH);

always @* begin
    next = state;
    case (state)
        S_WAIT:      if (frame_start && pixel_valid) next = S_INGEST;
    endcase
end

```

```

    S_INGEST:    if (fb_frame_done) next = S_INIT_SCAN;
    S_INIT_SCAN: next = S_PE_START;
    S_PE_START:  next = S_PE_WAIT;
    S_PE_WAIT:   if (pe_done) next = S_C1_START;
    S_C1_START:  next = S_C1_WAIT;
    S_C1_WAIT:   if (c1_done) next = S_C2_START;
    S_C2_START:  next = S_C2_WAIT;
    S_C2_WAIT:   if (c2_done) next = S_C3_START;
    S_C3_START:  next = S_C3_WAIT;
    S_C3_WAIT:   if (c3_done) next = S_FC_START;
    S_FC_START:  next = S_FC_WAIT;
    S_FC_WAIT:   if (fc_done) next = S_SCORE;
    S_SCORE: begin
        if (scan_x_last && scan_y_last) next = S_LATCH;
        else                             next = S_PE_START;
    end
    S_LATCH:     next = S_WAIT;
    default:     next = S_WAIT;
endcase
end

always @(posedge clk) begin
    if (rst) begin
        state      <= S_WAIT;
        scan_x     <= 8'd0;
        scan_y     <= 7'd0;
        best_conf  <= 32'sd0;
        best_valid <= 1'b0;
        best_face_x <= 8'd0;
        best_face_y <= 7'd0;
        best_face_w <= 8'd72;
        best_face_h <= 7'd72;
        face_valid <= 1'b0;
        face_x     <= 8'd0;
        face_y     <= 7'd0;
        face_w     <= 8'd72;
        face_h     <= 7'd72;
        scan_done  <= 1'b0;
        pe_start   <= 1'b0;
        c1_start   <= 1'b0;
        c2_start   <= 1'b0;
        c3_start   <= 1'b0;
        fc_start   <= 1'b0;
    end else begin
        state      <= next;
        scan_done  <= 1'b0;
        pe_start   <= 1'b0;
        c1_start   <= 1'b0;
        c2_start   <= 1'b0;
    end
end

```

```

c3_start <= 1'b0;
fc_start <= 1'b0;

case (state)
    S_INIT_SCAN: begin
        scan_x      <= 8'd0;
        scan_y      <= 7'd0;
        best_conf    <= THRESHOLD;
        best_valid   <= 1'b0;
    end

    S_PE_START: pe_start <= 1'b1;
    S_C1_START: c1_start <= 1'b1;
    S_C2_START: c2_start <= 1'b1;
    S_C3_START: c3_start <= 1'b1;
    S_FC_START: fc_start <= 1'b1;

    S_SCORE: begin
        if (fc_conf > best_conf) begin
            best_conf    <= fc_conf;
            best_valid   <= 1'b1;
            best_face_x  <= frame_face_x;
            best_face_y  <= frame_face_y;
            best_face_w  <= frame_face_w;
            best_face_h  <= frame_face_h;
        end
        if (scan_x_last) begin
            scan_x <= 8'd0;
            if (!scan_y_last) scan_y <= scan_y + STRIDE[6:0];
        end else begin
            scan_x <= scan_x + STRIDE[7:0];
        end
    end

    S_LATCH: begin
        face_valid <= best_valid;
        face_x     <= best_face_x;
        face_y     <= best_face_y;
        face_w     <= best_face_w;
        face_h     <= best_face_h;
        scan_done  <= 1'b1;
    end

    default: ;
endcase
end
end
endmodule

```

```
`default_nettype wire
```

Module Description :

The Detector Top Module implements the complete real-time face detection pipeline for the FPGA-based vision system.

The execution :

- receives grayscale image frames
- performs sliding-window scanning
- executes multi-layer CNN inference
- evaluates detection confidence
- outputs the highest-confidence face bounding box coordinates

Module : frame_buffer

Module code :

```
// holds one 160x120 grayscale frame
// write: streaming pixel_valid/frame_start
// read: random 15-bit addr, 1-cycle latency
// frame_done pulses on the last pixel of each frame

`default_nettype none

module frame_buffer #(
    parameter FRAME_W = 160,
    parameter FRAME_H = 120,
    parameter ADDR_W = 15          // ceil(log2(160*120)) = 15
) (
    input wire      clk,
    input wire      rst,          // synchronous, active high

    // streaming write port
    input wire      pixel_valid,
    input wire      frame_start,
    input wire      line_start,   // unused but accepted (sync hint)
    input wire [7:0] pixel,
    output reg      frame_done,   // 1-cycle pulse at end of frame

    // random read port
    input wire [ADDR_W-1:0] r_addr,
    output reg [7:0]      r_data
);

    localparam FRAME_PIXELS = FRAME_W * FRAME_H;    // 19200

    reg [7:0]      mem [0:FRAME_PIXELS-1];
    reg [ADDR_W-1:0] wr_ptr;                        // next write address

    // frame_start resets write addr to 0, recovering from partial frames
    wire [ADDR_W-1:0] write_addr = (pixel_valid && frame_start)
                                    ? {ADDR_W{1'b0}}
                                    : wr_ptr;

    // suppress unused warning
    wire _unused_line_start = line_start;

    always @(posedge clk) begin
        if (rst) begin
            wr_ptr      <= {ADDR_W{1'b0}};
            frame_done <= 1'b0;
        end else begin
            frame_done <= 1'b0;
        end
    end
endmodule
```



```

        if (pixel_valid) begin
            mem[write_addr] <= pixel;
            if (frame_start) begin
                // wrote pixel 0, next goes to addr 1
                wr_ptr <= {{(ADDR_W-1){1'b0}}, 1'b1};
            end else begin
                wr_ptr <= wr_ptr + 1'b1;
                if (wr_ptr == FRAME_PIXELS - 1) begin
                    frame_done <= 1'b1;
                end
            end
        end
    end
end

// read port
always @(posedge clk) begin
    r_data <= mem[r_addr];
end

endmodule

`default_nettype wire

```

Module Description :

This is a 19,200-entry BRAM that holds a full 160x120 grayscale frame. On the write side, it monitors the pixel stream and uses frame_start to reset the write pointer to zero, which helps if we ever lose sync. For reading, it provides 1-cycle latency random access so the CNN layers can pull any pixel they need from the frame whenever they want.

Module : patch_extractor

Module code :

```
// pull a 24x24 patch from frame_buffer, convert uint8 → int8, write to input act_buffer
// DILATE stretches the patch to cover PATCH*DILATE pixels in the real frame

`default_nettype none

module patch_extractor #(
    parameter FRAME_W = 160,
    parameter PATCH   = 24
) (
    input  wire      clk,
    input  wire      rst,          // synchronous, active high
    input  wire      start,        // 1-cycle pulse to begin
    input  wire [7:0] patch_x,      // 0 .. (FRAME_W - PATCH*dilate_in)
    input  wire [6:0] patch_y,      // 0 .. (FRAME_H - PATCH*dilate_in)
    input  wire [2:0] dilate_in,    // 1..7; runtime sub-sample stride
    output reg       done,          // 1-cycle pulse when finished

    // frame_buffer read port (1-cycle latency)
    output reg [14:0] fb_r_addr,
    input  wire [7:0] fb_r_data,

    // act_buffer write port
    output reg       ab_we,
    output reg [9:0] ab_w_addr,
    output reg signed [7:0] ab_w_data
);

// FSM
localparam [1:0] S_IDLE  = 2'd0;
localparam [1:0] S_WALK  = 2'd1;
localparam [1:0] S_DRAIN = 2'd2;
localparam [1:0] S_FIN   = 2'd3;

reg [1:0] state, next;

// read counters – address being issued this cycle
reg [4:0] ky;      // 0..23 in 5 bits
reg [4:0] kx;      // 0..23

// delayed by one cycle for write-back
reg [4:0] ky_d;
reg [4:0] kx_d;
reg      valid_d; // we issued a read on the previous cycle

// latch inputs so caller can deassert after start
reg [7:0] saved_px;
```

```

reg [6:0] saved_py;
reg [2:0] saved_dilate;

wire kx_last  = (kx == PATCH - 1);
wire ky_last  = (ky == PATCH - 1);
wire walk_last = ky_last && kx_last;    // last read cycle

// next-state
always @* begin
    next = state;
    case (state)
        S_IDLE:  if (start)      next = S_WALK;
        S_WALK:  if (walk_last) next = S_DRAIN;
        S_DRAIN:                next = S_FIN;
        S_FIN:                next = S_IDLE;
        default:                next = S_IDLE;
    endcase
end

// sequential
always @(posedge clk) begin
    if (rst) begin
        state      <= S_IDLE;
        ky         <= 0; kx <= 0;
        ky_d       <= 0; kx_d <= 0;
        valid_d    <= 1'b0;
        done       <= 1'b0;
        ab_we      <= 1'b0;
        saved_px   <= 0;
        saved_py   <= 0;
        saved_dilate <= 3'd1;
    end else begin
        state <= next;
        done  <= (next == S_FIN);
        ab_we <= 1'b0;    // default each cycle

        case (state)
            S_IDLE: begin
                if (start) begin
                    saved_px    <= patch_x;
                    saved_py    <= patch_y;
                    saved_dilate <= (dilate_in == 3'd0) ? 3'd1 : dilate_in;
                    ky          <= 0;
                    kx          <= 0;
                    valid_d     <= 1'b0;
                end
            end
            S_WALK: begin

```

```

        // data from last cycle's read is ready; write it now
        if (valid_d) begin
            ab_we      <= 1'b1;
            ab_w_addr <= ky_d * PATCH + kx_d;
            // uint8 → int8 by flipping sign bit
            ab_w_data <= fb_r_data ^ 8'h80;

            end

        // save for next cycle's write
        ky_d    <= ky;
        kx_d    <= kx;
        valid_d <= 1'b1;

        // advance read counters
        if (kx_last) begin
            kx <= 0;
            if (!ky_last) ky <= ky + 1'b1;
        end else begin
            kx <= kx + 1'b1;
        end
    end

    S_DRAIN: begin
        // drain the last read
        ab_we      <= 1'b1;
        ab_w_addr <= ky_d * PATCH + kx_d;
        ab_w_data <= fb_r_data ^ 8'h80;
    end

    default: ;
endcase
end

// address driver - combinational
always @* begin
    fb_r_addr = (saved_py + ky*saved_dilate) * FRAME_W
                + (saved_px + kx*saved_dilate);
end

endmodule

`default_nettype wire

```

Module Description :

This module extracts a 24×24 pixel patch from the main frame buffer and stores it into the activation buffer for CNN processing. It also converts unsigned grayscale pixels into signed 8-bit activation values and supports runtime dilation scaling,

allowing the detector to scan larger image regions without increasing CNN input size.

Module : act_buffer

Module code :

```
// simple dual-port BRAM for inter-layer activations
// write port: synchronous. read port: 1-cycle latency

`default_nettype none

module act_buffer #(
    parameter WIDTH  = 8,
    parameter DEPTH  = 576,
    parameter ADDR_W = 10
) (
    input  wire          clk,

    // write port
    input  wire          we,
    input  wire [ADDR_W-1:0] w_addr,
    input  wire signed [WIDTH-1:0] w_data,

    // read port, 1-cycle latency
    input  wire [ADDR_W-1:0] r_addr,
    output reg signed [WIDTH-1:0] r_data
);

    reg [WIDTH-1:0] mem [0:DEPTH-1];

    always @(posedge clk) begin
        if (we) mem[w_addr] <= w_data;
        r_data <= mem[r_addr];
    end

endmodule

`default_nettype wire
```

Module Description :

This module stores intermediate neural network activation data between processing layers. It functions as a simple dual-port block RAM (BRAM) that supports synchronous write operations and one-cycle-latency read operations.

Module : weight_rom

Module code :

```
// synchronous ROM for weights and biases, 1-cycle read latency
// initialized from a hex file
// set MEM_FILE="" to skip loading for testbenches

`default_nettype none

module weight_rom #(
    parameter WIDTH      = 8,
    parameter DEPTH      = 256,
    parameter ADDR_W     = 8,
    parameter MEM_FILE = ""
) (
    input  wire          clk,
    input  wire [ADDR_W-1:0]  addr,
    output reg  signed [WIDTH-1:0] data
);

    reg [WIDTH-1:0] mem [0:DEPTH-1];

    initial begin
        if (MEM_FILE != "") begin
            $readmemh(MEM_FILE, mem);
        end
    end

    always @(posedge clk) begin
        data <= mem[addr];
    end

endmodule

`default_nettype wire
```

Module Description :

This module implements a synchronous ROM for storing CNN weights and bias parameters. Memory values are initialized from external hexadecimal files, enabling trained model parameters to be loaded directly into FPGA memory. The ROM provides 1-cycle read latency for convolution and fully connected layer inference.

Module : requantize

Module code :

```
// int32 acc → int8 post-relu
// must match quantize.py

`default_nettype none

module requantize #(
    parameter SHIFT = 8          // requantization right-shift, signed integer
) (
    input  wire signed [31:0] acc,
    output wire signed [7:0]  q
);

    localparam signed [31:0] BIAS = (SHIFT > 0) ? (32'sd1 <<< (SHIFT - 1)) : 32'sd0;

    wire signed [31:0] shifted;
    generate
        if (SHIFT > 0) begin : g_rshift
            assign shifted = (acc + BIAS) >>> SHIFT;
        end else if (SHIFT == 0) begin : g_noshift
            assign shifted = acc;
        end else begin : g_lshift
            assign shifted = acc <<< (-SHIFT);
        end
    endgenerate

    // relu + saturate to [0, 127]
    assign q = (shifted <= 32'sd0)    ? 8'sd0    :
               (shifted >= 32'sd127) ? 8'sd127 :
               shifted[7:0];

endmodule

`default_nettype wire
```

Module Description :

This module converts a 32-bit accumulated CNN output into an 8-bit quantized activation value. The accumulated result is rescaled using a configurable arithmetic shift operation, followed by ReLU activation and saturation to the range of 0–127. The implementation is designed to match the quantization behavior used during Python model training and export.

Module : conv_layer

Module code :

```
// generic 2D conv + bias + relu + requantize. one mac per cycle
// address layout: input [ic,iy,ix], weight [oc,ic,ky,kx], output [oc,oy,ox]

`default_nettype none

module conv_layer #(
    parameter IN_CH          = 1,
    parameter OUT_CH         = 8,
    parameter IN_H           = 24,
    parameter IN_W           = 24,
    parameter K              = 3,
    parameter STRIDE         = 2,
    parameter OUT_H          = (IN_H - K) / STRIDE + 1,
    parameter OUT_W          = (IN_W - K) / STRIDE + 1,
    parameter REQUANT_SHIFT = 8
) (
    input  wire      clk,
    input  wire      rst,          // synchronous, active high
    input  wire      start,        // 1-cycle pulse to begin
    output reg       done,         // high for 1 cycle when finished

    // input activation memory (read port, 1-cycle latency)
    output reg [$clog2(IN_CH*IN_H*IN_W)-1:0] in_addr,
    input  wire signed [7:0] in_data,

    // weight ROM (read port, 1-cycle latency)
    output reg [$clog2(OUT_CH*IN_CH*K*K)-1:0] w_addr,
    input  wire signed [7:0] w_data,

    // bias ROM (read port, 1-cycle latency)
    output reg [$clog2(OUT_CH)-1:0] b_addr,
    input  wire signed [31:0] b_data,

    // output activation memory (write port)
    output reg out_we,
    output reg [$clog2(OUT_CH*OUT_H*OUT_W)-1:0] out_addr,
    output reg signed [7:0] out_data
);

// counters guard
localparam OC_BITS = (OUT_CH > 1) ? $clog2(OUT_CH) : 1;
localparam OY_BITS = (OUT_H > 1) ? $clog2(OUT_H) : 1;
localparam OX_BITS = (OUT_W > 1) ? $clog2(OUT_W) : 1;
localparam IC_BITS = (IN_CH > 1) ? $clog2(IN_CH) : 1;
localparam K_BITS  = (K > 1) ? $clog2(K) : 1;
```

```

reg [OC_BITS-1:0] oc;
reg [OY_BITS-1:0] oy;
reg [OX_BITS-1:0] ox;
reg [IC_BITS-1:0] ic;
reg [K_BITS-1:0] ky;
reg [K_BITS-1:0] kx;

// last-iteration flags
wire ic_last = (ic == IN_CH - 1);
wire ky_last = (ky == K - 1);
wire kx_last = (kx == K - 1);
wire ox_last = (ox == OUT_W - 1);
wire oy_last = (oy == OUT_H - 1);
wire oc_last = (oc == OUT_CH - 1);

wire kernel_last = ic_last & ky_last & kx_last;

// FSM: IDLE → READ_BIAS → LOAD_BIAS → ISSUE → MAC → WB → FIN
localparam [2:0] S_IDLE      = 3'd0;
localparam [2:0] S_READ_BIAS = 3'd1;
localparam [2:0] S_LOAD_BIAS = 3'd2;
localparam [2:0] S_ISSUE     = 3'd3;
localparam [2:0] S_MAC       = 3'd4;
localparam [2:0] S_WB        = 3'd5;
localparam [2:0] S_FIN       = 3'd6;

reg [2:0] state, next;

reg signed [31:0] acc;

// 8x8 signed product
wire signed [15:0] mac_prod = in_data * w_data;

// Requantize unit (combinational)
wire signed [7:0] q_out;
requantize #(.SHIFT(REQUANT_SHIFT)) u_rq (.acc(acc), .q(q_out));

// next-state
always @* begin
    next = state;
    case (state)
        S_IDLE:      if (start) next = S_READ_BIAS;
        S_READ_BIAS: next = S_LOAD_BIAS;
        S_LOAD_BIAS: next = S_ISSUE;
        S_ISSUE:     next = S_MAC;
        S_MAC: begin
            if (kernel_last) next = S_WB;
            else              next = S_ISSUE;
        end
    end
end

```

```

        S_WB: begin
            if (oc_last && oy_last && ox_last) next = S_FIN;
            else
                next = S_READ_BIAS;
            end
        S_FIN:      next = S_IDLE;
        default:    next = S_IDLE;
    endcase
end

// sequential
always @(posedge clk) begin
    if (rst) begin
        state      <= S_IDLE;
        oc         <= 0; oy <= 0; ox <= 0;
        ic         <= 0; ky <= 0; kx <= 0;
        acc        <= 32'sd0;
        done       <= 1'b0;
        out_we     <= 1'b0;
    end else begin
        state <= next;
        done  <= (next == S_FIN);
        out_we <= 1'b0;

        case (state)
            S_IDLE: begin
                oc <= 0; oy <= 0; ox <= 0;
                ic <= 0; ky <= 0; kx <= 0;
            end

            // bias from previous cycle's read
            S_LOAD_BIAS: begin
                acc <= b_data;
                ic  <= 0; ky <= 0; kx <= 0;
            end

            S_MAC: begin
                // sign-extend 16-bit product to 32
                acc <= acc + {{16{mac_prod[15]}}, mac_prod};

                // advance kernel counters
                if (kx_last) begin
                    kx <= 0;
                    if (ky_last) begin
                        ky <= 0;
                        if (!ic_last) ic <= ic + 1'b1;
                    end else begin
                        ky <= ky + 1'b1;
                    end
                end else begin
                    end else begin

```

```

        kx <= kx + 1'b1;
    end
end

    S_WB: begin
        // write requantized output
        out_we <= 1'b1;
        out_addr <= oc * (OUT_H * OUT_W) + oy * OUT_W + ox;
        out_data <= q_out;
        // advance position
        if (ox_last) begin
            ox <= 0;
            if (oy_last) begin
                oy <= 0;
                if (!oc_last) oc <= oc + 1'b1;
            end else begin
                oy <= oy + 1'b1;
            end
        end else begin
            ox <= ox + 1'b1;
        end
    end
end

    default: ;
endcase
end

end

// address drivers
always @* begin
    b_addr = oc;

    // addresses for next mac
    in_addr = ic * (IN_H * IN_W)
        + (oy * STRIDE + ky) * IN_W
        + (ox * STRIDE + kx);
    w_addr = oc * (IN_CH * K * K)
        + ic * (K * K)
        + ky * K
        + kx;
end

endmodule

`default_nettype wire

```

Module Description :

This module performs two-dimensional convolution operations for neural network inference.

The module executes:

- convolution
- bias addition
- ReLU activation
- requantization

using a sequential multiply-accumulate (MAC) architecture optimized for FPGA implementation.

Module : fc_layer

Module code :

```
`default_nettype none

module fc_layer #(
    parameter IN_LEN  = 144,
    parameter OUT_LEN = 5
) (
    input  wire      clk,
    input  wire      rst,
    input  wire      start,
    output reg       done,

    // input act_buffer
    output wire [7:0] in_addr,
    input  wire signed [7:0] in_data,

    // weight ROM
    output wire [9:0] w_addr,
    input  wire signed [7:0] w_data,

    // bias ROM
    output wire [2:0] b_addr,
    input  wire signed [31:0] b_data,

    // outputs
    output reg signed [31:0] out_conf,
    output reg signed [31:0] out_x0,
    output reg signed [31:0] out_y0,
    output reg signed [31:0] out_w,
    output reg signed [31:0] out_h
);

// FSM States
localparam [2:0] S_IDLE      = 3'd0;
localparam [2:0] S_READ_BIAS = 3'd1;
localparam [2:0] S_LOAD_BIAS = 3'd2;
localparam [2:0] S_MAC       = 3'd3;
localparam [2:0] S_WB        = 3'd4;
localparam [2:0] S_FIN        = 3'd5;

reg [2:0] state, next;
reg [2:0] oc;
reg [7:0] i;
reg signed [31:0] acc;
reg pipe_vld;

// --- FIX: Explicitly declare the multiplier wire ---
```

```

wire signed [15:0] mac_prod = in_data * w_data;

// FSM Next State Logic
always @* begin
    next = state;
    case (state)
        S_IDLE:      if (start) next = S_READ_BIAS;
        S_READ_BIAS: next = S_LOAD_BIAS;
        S_LOAD_BIAS: next = S_MAC;
        S_MAC:        if (i == IN_LEN-1 && pipe_vld) next = S_WB;
                     else next = S_MAC;
        S_WB:          if (oc == OUT_LEN-1) next = S_FIN;
                     else next = S_READ_BIAS;
        S_FIN:          next = S_IDLE;
        default:        next = S_IDLE;
    endcase
end

// Sequential Logic
always @(posedge clk) begin
    if (rst) begin
        state    <= S_IDLE;
        oc       <= 0;
        i        <= 0;
        acc      <= 0;
        pipe_vld <= 0;
        done     <= 0;
        out_conf <= 0; out_x0 <= 0; out_y0 <= 0; out_w <= 0; out_h <= 0;
    end else begin
        state <= next;
        done  <= (next == S_FIN);

        case (state)
            S_IDLE: begin
                oc <= 0;
            end

            S_LOAD_BIAS: begin
                acc <= b_data;
                i <= 0;
                pipe_vld <= 0;
            end

            S_MAC: begin
                // Accumulate data from previous cycle's address
                if (pipe_vld) begin
                    acc <= acc + $signed(mac_prod);
                end
            end
        endcase
    end
end

```

```

        if (i < IN_LEN - 1) begin
            i <= i + 1;
            pipe_vld <= 1;
        end else begin
            // Staying at i=143 for one last cycle to let pipe_vld catch the
last data
        end
    end

    S_WB: begin
        case (oc)
            3'd0: out_conf <= acc;
            3'd1: out_x0  <= acc;
            3'd2: out_y0  <= acc;
            3'd3: out_w   <= acc;
            3'd4: out_h   <= acc;
        endcase
        if (oc < OUT_LEN - 1) oc <= oc + 1;
    end
endcase
end
end

// Continuous Assignments for Addresses
assign b_addr = oc;
assign in_addr = i;
assign w_addr = (oc * 144) + i; // Explicitly using 144 for clarity

endmodule

```

Module Description :

This module performs the final neural network inference stage for face detection and bounding-box regression.

The module computes:

- detection confidence
- face X coordinate
- face Y coordinate
- face width
- face height

using a sequential multiply-accumulate (MAC) architecture optimized for FPGA implementation.

The layer processes 144 input features and produces 5 signed 32-bit output values corresponding to the final detection results.

Challenge Faced

1. Hard to implement from scratch
2. Learning a new camera module
3. Learning a new communication protocol
4. Handling different clock domains
5. Learning how to make a CNN model and understanding how it works
6. Dealing with a convolution layer latency (from computing) problem.
7. Resource management in a limited space (due to Basys3), from storing image data to weights and biases (convolution layer)
8. Quantizing a floating point, from model training, to an integer for an FPGA's storage.

AI Usage Declaration

- Uses Claude Code for generating testbenches, README.md, project structure, consulting, and CNN model creation and implementation.
- Uses Gemini for debugging.